Archive logs and redo logs

MANAGING THE REDO LOG

---

1. PURPOSE OF REDO LOGS
   Redo logs record **all changes made to the database**.
   These changes are called **redo entries**.

Redo entries are:

- Generated whenever data changes
- First stored in memory (Redo Log Buffer)
- Later written to disk (Online Redo Log Files)

Redo logs are critical for:

- Instance recovery after crashes
- Normal database operation (database cannot run without them)

If online redo logs are missing or deleted → **database will crash**

REDO LOG ARCHITECTURE (BASIC FLOW)

- User makes a data change
- Redo entry is generated
- Redo entry is stored in **Redo Log Buffer (memory)**
- **LGWR (Log Writer)** writes redo entries to **Online Redo Log Files (disk)**

Redo logs are written **sequentially**, not randomly.

REDO LOG GROUPS AND CIRCULAR WRITING

- Online redo logs are divided into **groups**
- Each group has **one or more members**
- Minimum required groups: **2**
- Best practice (production): **3 or more groups**

LGWR writes redo logs in a **circular fashion**:

- Group 1 → Group 2 → Group 3 → back to Group 1
- File sizes are **fixed** and do not grow automatically

This circular reuse is normal behavior.

REDO LOG GROUP STATUS AND LOG SWITCH

Redo log group states:

- CURRENT / ACTIVE → group currently being written to
- INACTIVE → not currently used

Process:

- LGWR writes to current group until it is full
- LGWR switches to the next group
- This is called a **LOG SWITCH**
- Previous group becomes inactive (or active but not current)

NOARCHIVELOG MODE

Definition:

- Redo entries exist **only in online redo log files**
- Old redo entries are overwritten during reuse

Characteristics:

- No point-in-time recovery
- Cannot recover to a past state
- Cannot use Data Guard or advanced HA features
- Suitable for test or non-critical databases

Recovery capability:

- Only **instance recovery**
- No media recovery

ARCHIVELOG MODE (CONCEPTUAL INTRO)

Key difference:

- Redo logs are **copied before reuse**

What happens during a log switch:

- LGWR moves to next redo log group
- **ARCn (Archiver process)** copies the full redo log
- Copy is saved as an **Archived Redo Log file**

Archived redo logs:

- Generated sequentially
- Stored separately from online redo logs
- Used only for recovery
- Do NOT affect normal database operations

Important:

- Oracle does NOT manage archive space
- DBA must delete old archive logs manually

REDO LOG MEMBERS AND MULTIPLEXING

Redo log structure:

- Each group contains **multiple members**
- Members inside a group are **identical**
- This is called **Redo Log Multiplexing**

Purpose:

- High availability
- Protection against disk failure

Best practice:

- Place members of the same group on **different disks**
- Ideally on **different storage controllers**

Example:

- Group 1 → G1_1 on Disk 1, G1_2 on Disk 2
- Group 2 → G2_1 on Disk 1, G2_2 on Disk 2

If Disk 1 fails → database continues using Disk 2

Note:

- Oracle does not know disk topology
- DBA decides member placement

REDO MULTIPLEXING VS STORAGE MIRRORING

Some DBAs argue:

- If storage already supports mirroring (ASM, RAID), multiplexing is optional

Reality:

- Most production systems still multiplex redo logs
- Considered a **safe and conservative DBA practice**

ONLINE REDO LOG FILE MANAGEMENT RULES

- Cannot modify the CURRENT redo log group
- Cannot drop a redo log group in use
- Redo log sizes should be consistent across groups
- Frequent log switches indicate redo logs are too small

REDO LOGS VS UNDO (HIGH-LEVEL COMPARISON)

Redo:

- Records **how data changes**
- Used for crash and instance recovery
- Applies changes forward

Undo:

- Records **how to reverse changes**
- Used for rollback and read consistency
- Applies changes backward

Both are mandatory but serve different purposes.

IMPORTANT DYNAMIC PERFORMANCE VIEWS

To view redo log configuration:

- V$LOG
  Shows:
  - Group number
  - Status (CURRENT, ACTIVE, INACTIVE)
  - Size
- V$LOGFILE
  Shows:
  - Member file names
  - Group associations
  - File locations

These views are essential for redo log troubleshooting.

MANAGING REDO LOG GROUPS AND MEMBERS

MAXLOGFILES LIMITATION

- MAXLOGFILES controls the **maximum number of redo log groups** in a database
- It is **not an initialization parameter**
- Defined **only at database creation time**
- Specified in the `CREATE DATABASE` statement
- **Cannot be changed later**

Defaults:

- Databases created using **DBCA** usually have `MAXLOGFILES = 16`
- In practice, production databases rarely exceed **3–4 redo log groups**

How to check MAXLOGFILES:

- Query `V$CONTROLFILE_RECORD_SECTION`

Purpose:

- Prevents uncontrolled growth of redo log groups
- Enforced by control file metadata

REDO LOG GROUP SIZE CONSISTENCY

- All redo log members within a group must be **same size**
- Across groups:
  - Temporary size differences are allowed
  - Final configuration should have **uniform group sizes**
- Best practice:
  - Drop groups with unwanted sizes
  - Keep only groups with required size

CREATING A REDO LOG GROUP

Command:

- `ALTER DATABASE ADD LOGFILE`

Key points:

- Provide full path names of group members
- Specify size (applies to all members)
- Group name format:
  - `GROUP n` (n is sequential)

Automatic group numbering:

- If group number is not specified:
  - Oracle assigns the next available number
  - Example: existing GROUP 1–3 → new group becomes GROUP 4

Manual group numbering:

- You may explicitly specify `GROUP n` after `LOGFILE`

Metadata storage:

- Redo log file names and locations are stored in the **control files**

OMF AND REDO LOG FILES

Parameter:

- `DB_CREATE_FILE_DEST`
  - Default location for **datafiles**
  - Also applies to **redo log files**

Overriding redo log locations:

- Use `DB_CREATE_ONLINE_LOG_DEST_n`
  - n = 1 to 5
  - Allows **redo log multiplexing**

Example behavior:

- If `DB_CREATE_ONLINE_LOG_DEST_1` and `_2` are set:
  - Oracle automatically creates **two members per redo group**
  - Members are placed in both locations

Important:

- When `DB_CREATE_ONLINE_LOG_DEST_n` is set:
  - `ALTER DATABASE ADD LOGFILE` does NOT require file names
  - Oracle auto-generates names and locations

Practical DBA note:

- OMF for redo logs is **optional**
- Redo logs are rarely created or modified
- OMF is more critical for datafiles than redo logs

ADDING A REDO LOG MEMBER TO A GROUP

Use cases:

- Increase redundancy
- Replace a lost redo log member

Rules before adding:

- Target group must NOT be the **CURRENT** group
- Check status using `V$LOG`
- If CURRENT:
  - Force log switch:
    - `ALTER SYSTEM SWITCH LOGFILE`
  - Wait until group becomes INACTIVE

Command:

- `ALTER DATABASE ADD LOGFILE MEMBER 'file_name' TO GROUP n`

Important rules:

- Size is NOT specified
- New member inherits size from existing members
- File name must be specified (even with OMF)
- Cannot add member if:
  - All members of the group are lost
  - At least one valid member must exist

REDO LOG CONFIGURATION USING DBCA

DBCA option:

- “Customize Storage Locations”

Behavior:

- Default DBCA setup:
  - 3 redo log groups
  - Each group size: **200 MB**

Practical guidance:

- 200 MB is small for modern OLTP systems
- General recommendation:
  - Increase to **1 GB**
- Final size depends on:
  - Transaction rate (TPS)
  - Workload type

Read-mostly systems:

- Smaller redo logs are acceptable

FRA AND REDO LOG MULTIPLEXING

Observation:

- DBCA UI may show only one member per group
- If **FRA is enabled**:
  - Redo logs are actually multiplexed
  - One member in specified location
  - Second member automatically placed in FRA

RELOCATING REDO LOG MEMBERS

Common reason:

- Storage issues
- Hardware migration

Procedure:

1. Shutdown database
2. Copy redo log files to new location
3. Startup database in **MOUNT** mode
4. Execute:
    - `ALTER DATABASE RENAME FILE old_name TO new_name`
5. Open database

Important:

- Original redo log files must be **available and valid**
- Control file is updated during rename

DROPPING REDO LOG GROUPS

Rules:

- Database must have **at least two redo log groups**
- If only two exist:
  - Add a new group first
- Group must be **INACTIVE**
- In ARCHIVELOG mode:
  - Group must be archived

Steps:

- If CURRENT:
  - Perform log switch
- Drop group:
  - `ALTER DATABASE DROP LOGFILE GROUP n`

This removes:

- The group
- All its members

DROPPING REDO LOG MEMBERS

Command:

- `ALTER DATABASE DROP LOGFILE MEMBER 'full_path_name'`

Rules:

- Cannot drop the last remaining member of a group
- Group must retain at least one valid member

FORCING A LOG SWITCH

Command:

- `ALTER SYSTEM SWITCH LOGFILE`

Usage:

- Testing
- Maintenance
- Preparing redo logs for archival
- Making groups inactive for maintenance

REDO LOGS IN MULTITENANT (CDB / PDB)

Key points:

- Redo logs belong to the **CDB**
- Shared by all PDBs
- Cannot manage redo logs from a PDB

Rules:

- Must connect to **CDB$ROOT**
- Redo generation is usually higher in multitenant OLTP systems
- Redo log groups are typically **large (multi-GB)**

UNDO VS REDO — CORE DBA CONCEPTS

FUNDAMENTAL DIFFERENCE (INTENTION)

Undo data:

- Purpose: **reverse changes**
- Answers the question: *“How do I undo what just happened?”*

Redo data:

- Purpose: **reproduce changes**
- Answers the question: *“How do I reapply what happened?”*

They serve **opposite but complementary roles**.

WHAT UNDO DATA IS USED FOR

Undo data supports:

- ROLLBACK operations
- READ CONSISTENCY
- FLASHBACK operations

Undo ensures:

- Users do not see **inconsistent data**
- Queries can read **older committed versions** of data

Undo protects against:

- Inconsistent reads during concurrent transactions

Undo storage:

- Stored in **UNDO tablespaces**
- Inside **undo segments**

WHAT REDO DATA IS USED FOR

Redo data supports:

- Media recovery
- Instance crash recovery
- Replication (Data Guard, Streams, etc.)

Redo ensures:

- Changes are **never lost**
- Database can be recovered after failures

Redo storage:

- Online redo log files
- Archived redo log files (if ARCHIVELOG mode enabled)

UNDO AND REDO IN INSTANCE CRASH RECOVERY

Both are used together during instance recovery:

Phase 1: Roll Forward

- Uses **redo**
- Reapplies all changes from redo logs

Phase 2: Roll Back

- Uses **undo**
- Reverses uncommitted transactions

Key idea:

- Redo moves data **forward**
- Undo moves data **backward**

Together:

- They guarantee **database consistency and recoverability**

PRACTICAL DBA REALITY

- If database planning is done correctly:
  - Redo logs are rarely modified in production
- At database creation:
  - Redo log size is chosen for current + future workload
  - Redo logs are placed on high-performance storage
- Changes are needed only if:
  - Redo logs are lost
  - Redo logs are too small for workload

REDO LOG BEST PRACTICES

NUMBER OF REDO LOG GROUPS

Production databases:

- Minimum: **3 redo log groups**
- Recommended: **4 redo log groups** (if storage allows)

Reason:

- Prevents frequent waits
- Allows smoother log switching

REDO LOG MULTIPLEXING

If multiple storage mounts exist:

- Place members of the same group on **different mount points**
- Ideally, each mount is on a **different controller**

Goal:

- Avoid single point of failure

If only one mount exists:

- Storage must support:
  - Mirroring
  - Striping
- Use:
  - RAID
  - ASM

REDO LOG SIZE BEST PRACTICE

General rule:

- Log switches should not occur **more frequently than every 15 minutes**

If log switches occur:

- More frequently than 15 minutes → increase redo log size

Exceptions:

- Occasional 10-minute switches due to batch jobs are acceptable

Key point:

- This is a guideline, not a strict rule

REDO LOG PERFORMANCE CONSIDERATION

- Redo logs are written **continuously**
- Written by **LGWR**
- High latency storage → poor database performance

Therefore:

- Always place redo logs on **highest-performance storage**
- More critical than datafile storage performance

ARCHIVELOG MODE IN PRODUCTION

Production databases:

- Almost always run in **ARCHIVELOG mode**

Why:

- Enables point-in-time recovery
- Required for backups, Data Guard, replication

NOARCHIVELOG:

- Suitable only for test or non-critical systems

Managing Archived Redo Log Files

ARCHIVELOG Mode Overview

- **ARCHIVER process (ARCn):** Reads online redo log files and writes entries into archived redo logs.
- **Sequential creation:** New archived redo log file generated for every log switch.
- **Purpose:** Used for recovery and replication; not required for normal database operations.
- **Production use:** Most production databases run in ARCHIVELOG mode.

NOARCHIVELOG vs ARCHIVELOG

Feature	NOARCHIVELOG	ARCHIVELOG
Protection	Against instance crash only	Against instance crash & media failure
Media failure recovery	Not supported	Supported with backups
Online backups	Not possible	Possible
Management complexity	Minimal	Must manage archived redo logs
Storage requirement	Minimal	Must plan and provide storage for archived redo logs
Performance overhead	None	Slight, due to ARCn process and I/O streams
Typical use	POC/testing	Production

Configuring Archive Destinations

- **Destination types:**
  1. Directory path
  2. First Recovery Area (FRA) → automatically organized into subdirectories by date
  3. Remote database service → used in Data Guard setups
- **Setting destinations options:**
  1. `LOG_ARCHIVE_DEST_n` → Recommended, supports up to 31 destinations
  2. `LOG_ARCHIVE_DEST` + `LOG_ARCHIVE_DUPLEX_DEST` → Legacy, supports only 2 destinations
  - Do **not** mix the two options
- **Dynamic behavior:**
  - If all archive destinations are disabled while in ARCHIVELOG mode, the database will hang during log switches until a valid destination is set.

Attributes for LOG_ARCHIVE_DEST_n

- **LOCATION:** Specifies directory path for archived redo logs
- **USE_DB_RECOVERY_FILE_DEST:** Saves logs in FRA subdirectories (preferred for organization)
- **ALTERNATE:** Designates alternate destination if primary fails
- **MAX_FAILURE:** Number of retries before switching to alternate

Example for alternate destination setup:

```
ALTER SYSTEM SET LOG_ARCHIVE_DEST_3='LOCATION=/backup/archive3';
ALTER SYSTEM SET LOG_ARCHIVE_DEST_2='LOCATION=/backup/archive2
MAX_FAILURE=1 ALTERNATE=3';
ALTER SYSTEM SET LOG_ARCHIVE_DEST_STATE_3=ALTERNATE;
```

- **Effect:** DEST_3 is inactive but will become active if DEST_2 fails.

Best Practices

- Always plan multiple destinations for redundancy.
- Use FRA for organized storage.
- Set up alternate destinations to prevent database hang.
- Monitor storage to avoid full or unavailable destinations.

Archived Redo Log Files in Oracle

1. LOG_ARCHIVE_FORMAT Parameter

- Optional parameter used to define the naming format of archived redo log files.
- If **not set**, Oracle assigns a default format based on the OS.
- Can include a mix of characters and **variables** (all start with `%`):

Variable	Meaning
`%s`	Sequence number (increments with every log switch)
`%t`	Thread number (used in RAC environments)
`%d`	Database ID (unique per database)
`%f`	Activation ID (unique per redo log activation, used in Data Guard)
`%r`	Resetlogs ID (increments after a resetlogs operation)

- **Key point:** Include `%s`, `%t`, `%r` to ensure unique names.
- Uppercase variables produce **fixed-length, zero-padded values**.
- **Note:** If the archive destination is **FRA (Fast Recovery Area)**, this parameter is ignored—Oracle automatically ensures unique names.

Enabling ARCHIVELOG Mode

1. Shutdown database cleanly.
2. Startup in **MOUNT** state.
3. Enable archiving:
4. `ALTER DATABASE ARCHIVELOG;`
5. Disable archiving (if needed):
6. `ALTER DATABASE NOARCHIVELOG;`
7. Open database in **read/write** mode.

- **DBCA Option:** During database creation, the “Fast Recovery Option” allows enabling archiving and setting archive format/destinations.
- **Best practice:** Leave new databases in NOARCHIVELOG mode during creation; enable ARCHIVELOG after preparation.

Checking Archiving Mode

- **SQL*Plus command:**
- `ARCHIVE LOG LIST;`

  Displays: mode, archive destination, oldest online log sequence, next log to archive, current log.

- **SQL query:**
- `SELECT LOG_MODE FROM V$DATABASE;`

---

4. CDB vs Non-CDB

- Online redo logs exist only in the **root container**.
- Archived redo logs are generated the same way in CDBs and non-CDBs.
- **PDBs cannot control archiving mode**—it follows the root container.

Dynamic Performance Views

- Use views like `V$ARCHIVE_DEST`, `V$ARCHIVED_LOG`, and `V$ARCHIVE_PROCESSES` to monitor archiver processes and archived redo logs (useful in maintenance).

Best Practices for Managing Archived Redo Logs

1. Enable ARCHIVELOG mode **after database creation**.
2. Use ARCHIVELOG mode in **all production databases**.
3. Store archived logs on **high-capacity storage** (performance not critical).
4. Protect storage with **disk mirroring/RAID or ASM**; consider multiple archive destinations if protection isn't possible.
5. Implement **deletion policy**: backup archive logs and delete from disk to free space.
6. Prefer storing archived redo logs in **FRA** for better organization.